



 **MAESTRÍA EN CIENCIA DE DATOS**

Práctica 3

Pipeline de Clasificación con Weka

Construcción visual de un flujo de Machine Learning en KnowledgeFlow para clasificar el dataset Iris con tres algoritmos distintos

 **Elaborado por:** Miguel Deza
 **Programa:** Maestría en Ciencia de Datos
 **Curso:** Introducción a Ciencia de Datos
 **Docente:** Edgar Sarmiento Calisaya
 **Fecha:** 05 de septiembre de 2026

Índice

1. 💡 Introducción	2
2. 🎯 Objetivo	2
3. 🛠️ Herramientas y fuentes de datos	2
4. ⚙️ Desarrollo	2
4.1. 🔍 Exploración del dataset	2
4.2. 🏗️ Construcción del pipeline visual	5
4.3. ⚙️ Algoritmos de clasificación evaluados	5
4.4. 📊 Comparación de resultados	7
4.5. 🌳 Visualización del árbol J48	8
5. ⚠️ Hallazgo técnico	9
6. 📄 Evidencia de ejecución	9
7. ✅ Conclusiones	9
8. 🔗 Referencias	10

1 Introducción

Un proyecto de ciencia de datos no termina en limpiar datos: el siguiente paso suele ser construir un **modelo** que aprenda un patrón a partir de ejemplos ya etiquetados, para luego predecir la etiqueta de casos nuevos. A esto se le llama **clasificación**, uno de los problemas centrales del *Machine Learning*.

En esta práctica utilicé **Weka** (*Waikato Environment for Knowledge Analysis*), una herramienta de código abierto para minería de datos, y específicamente su entorno **KnowledgeFlow**: un editor visual donde en vez de escribir código se arrastran componentes y se conectan con flechas para armar un flujo de procesamiento, de forma parecida a como armé el flujo ETL de Power BI en la práctica anterior.

Trabajé con el dataset clásico **Iris** (Fisher, 1936): 150 flores de 3 especies distintas, descritas por 4 medidas numéricas. El objetivo fue construir un pipeline que cargue estos datos, los explore, entrene varios algoritmos de clasificación y compare qué tan bien predicen la especie correcta.

2 Objetivo

Objetivo

Entender las etapas de un proyecto de ciencia de datos a través de la construcción de un pipeline visual para clasificar un conjunto de datos usando Weka.

3 Herramientas y fuentes de datos

Para desarrollar esta práctica utilicé:

 **Weka 3.8.6** – <https://www.cs.waikato.ac.nz/ml/weka> (instalado en Linux vía AUR)

 **KnowledgeFlow Environment** – editor visual de pipelines de Weka

 **Dataset:** `iris.arff` – 150 instancias, 4 atributos numéricos + clase (**Iris-setosa**, **Iris-versicolor**, **Iris-virginica**)

4 Desarrollo

4.1 Exploración del dataset

Antes de clasificar, analicé los 4 atributos numéricos del dataset (`sepalength`, `sepalwidth`, `petallength`, `petalwidth`, todos de tipo **REAL**) junto con la clase (`class`, tipo **nominal**, 3 categorías). El dataset tiene **150 instancias**, sin valores nulos, con la clase perfectamente balanceada (50 por especie). Sí detecté **3 filas exactamente duplicadas**, un detalle normal en este dataset tan estudiado (singularidad).

Atributo	Rango	Media	Var.	Desv. Est.	Correl. con clase
sepal length	4.3 – 7.9	5.84	0.68	0.83	0.78
sepal width	2.0 – 4.4	3.05	0.19	0.43	-0.42
petal length	1.0 – 6.9	3.76	3.09	1.76	0.95
petal width	0.1 – 2.5	1.20	0.58	0.76	0.96

Cuadro 1: Estadísticas descriptivas de los 4 atributos numéricos del dataset *Iris*.

El siguiente gráfico deja ver con claridad que el tamaño del **pétalo** (largo y ancho) predice la especie mucho mejor que el tamaño del **sépalo** – un dato que después confirmé al revisar el árbol de decisión del clasificador J48.

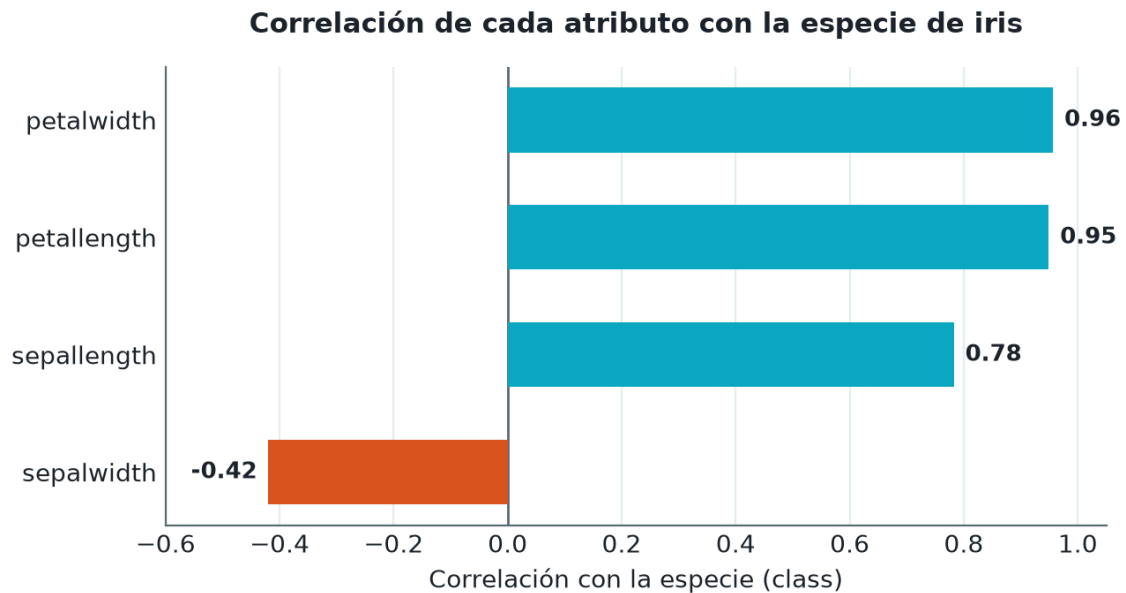


Figura 1: Correlación de cada atributo con la especie de iris. Los atributos del pétalo dominan claramente sobre los del sépalo.

Relaciones entre pares de atributos. Más allá de qué tan bien predice cada atributo la especie, también es útil ver cómo se relacionan los atributos *entre sí*. La matriz de correlación siguiente muestra que **petallength** y **petalwidth** están fuertemente correlacionados entre ellos (0.96) – tiene sentido, ambos crecen juntos según el tamaño general de la flor – y que **sepalwidth** es el único atributo que se correlaciona negativamente con todos los demás.

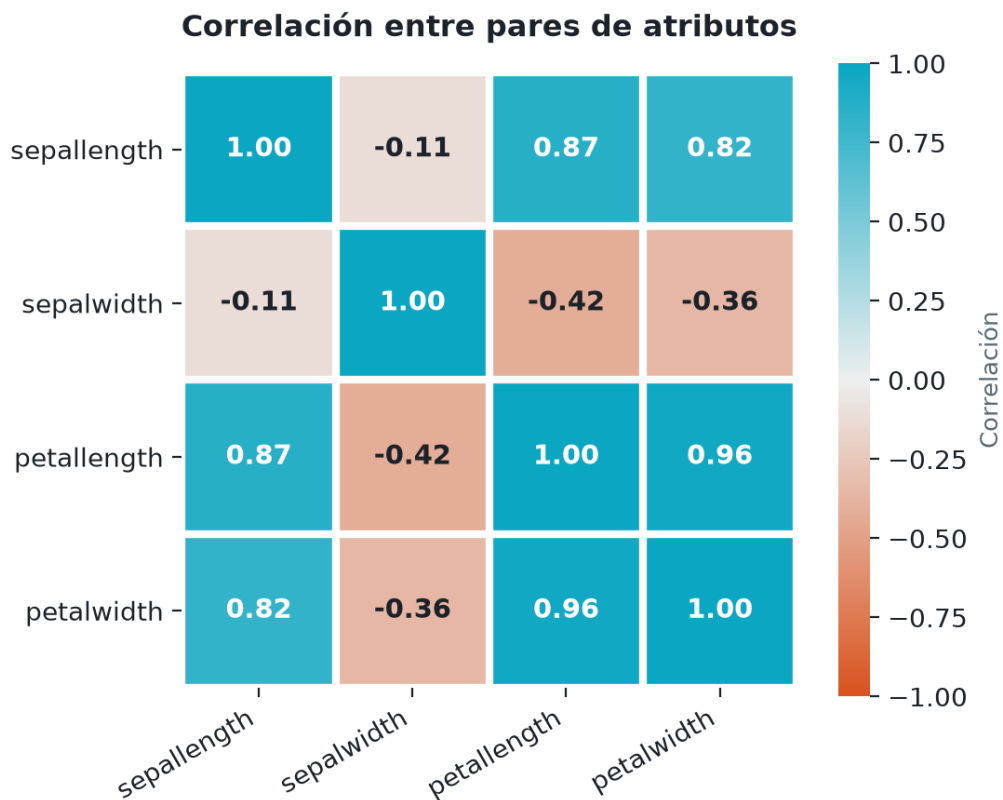


Figura 2: Matriz de correlación entre los 4 atributos numéricos (relaciones entre pares).

Distribución de valores por especie. Para entender mejor cómo se distribuyen los valores de los atributos clave (pétalo), separé sus valores por especie:

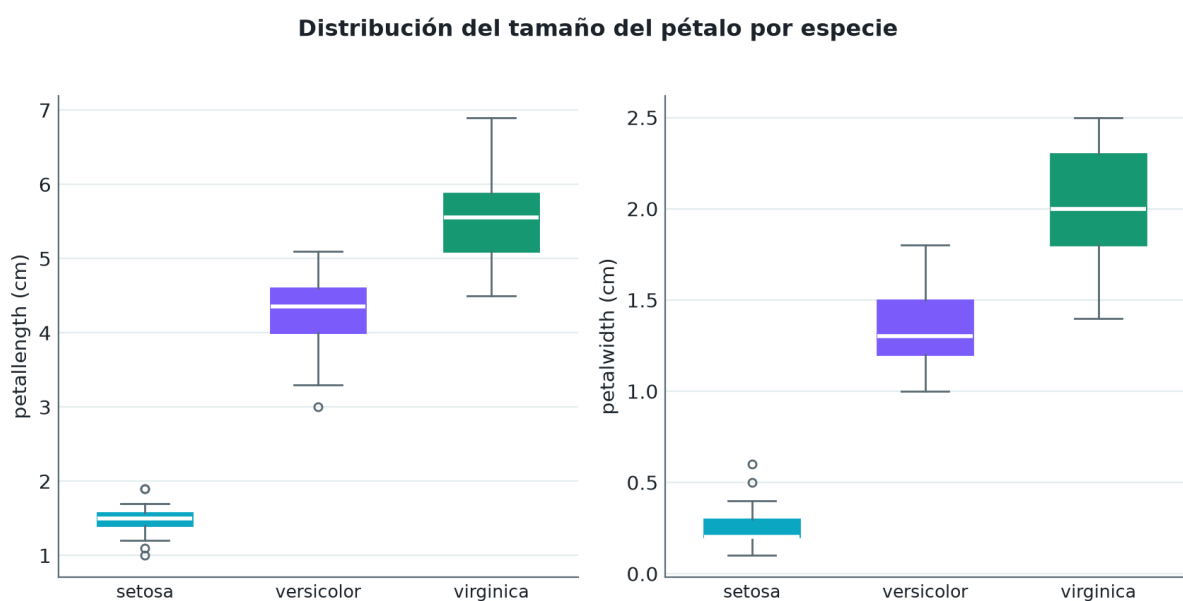


Figura 3: Distribución (mínimo, cuartiles, mediana y máximo) del largo y ancho de pétalo, por especie.

Estas cajas muestran visualmente lo mismo que ya sugería la correlación: *Iris-setosa* ocupa un rango de valores completamente separado del resto (sin solaparse ni un poco), mientras que las cajas de *versicolor* y *virginica* sí se superponen parcialmente – exactamente donde después aparecieron los errores de clasificación de los tres algoritmos.

4.2 Construcción del pipeline visual

En el KnowledgeFlow armé el siguiente flujo:

1. **ArffLoader** – cargó el archivo `iris.arff`.
2. **ClassAssigner** – designó la columna `class` como la etiqueta a predecir.
3. **CrossValidationFoldMaker** – dividió los datos en 10 pliegues para entrenar y evaluar de forma honesta (nunca examinando al modelo con los mismos datos que usó para aprender).
4. Tres ramas paralelas, una por cada **algoritmo de clasificación** (detallados en la siguiente sección), cada una con su propio **ClassifierPerformanceEvaluator** y **TextViewer** para revisar sus métricas por separado.

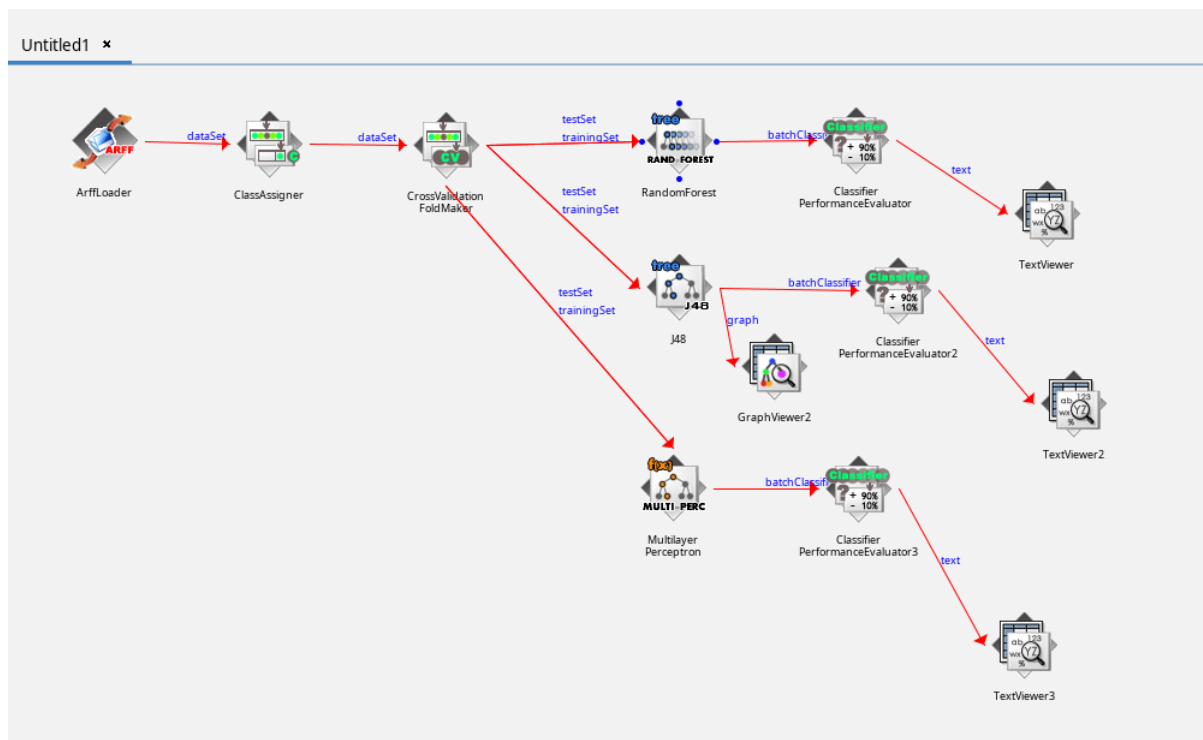


Figura 4: Pipeline completo armado en KnowledgeFlow: un origen de datos, validación cruzada, y tres algoritmos de clasificación evaluados en paralelo.

4.3 Algoritmos de clasificación evaluados

Ejecuté y comparé tres algoritmos con enfoques muy distintos:

- 🌲 **J48:** construye un único árbol de decisión, con reglas del tipo “si el pétalo mide menos de X, entonces es tal especie”.
- 🧠 **Random Forest:** entrena 100 árboles distintos sobre porciones aleatorias de los datos, y decide la especie por votación mayoritaria entre todos ellos.

- Multilayer Perceptron:** una red neuronal que ajusta sus pesos internos durante 500 iteraciones para aprender fronteras de decisión más complejas.

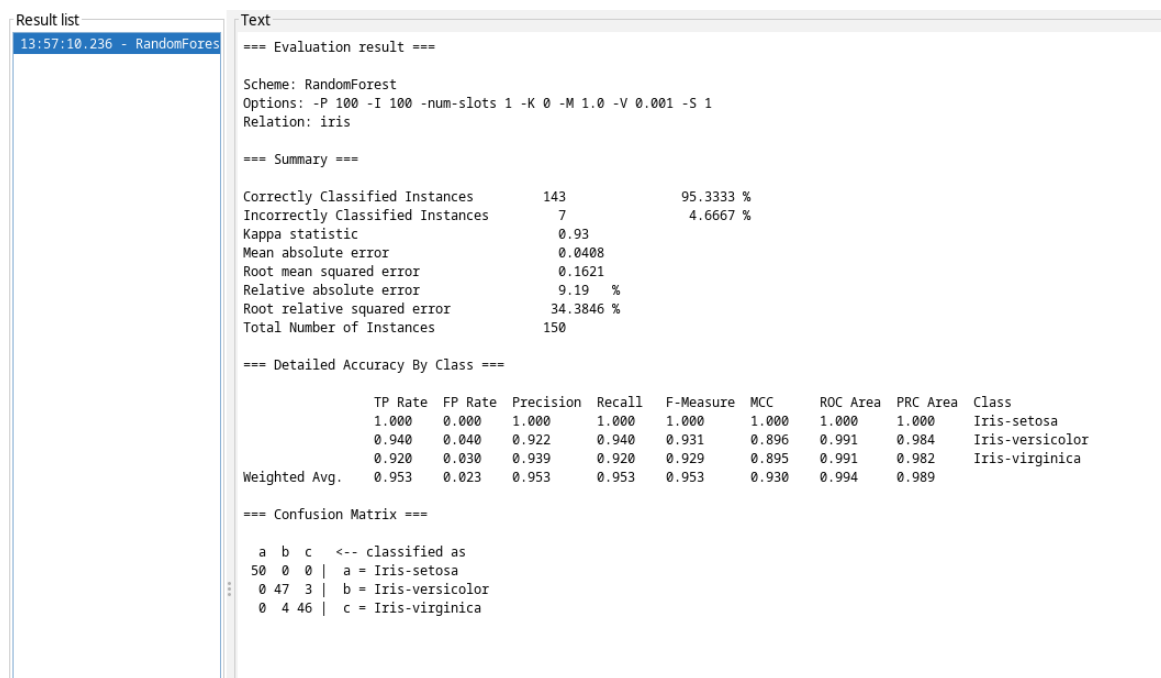


Figura 5: Resultado de Random Forest: 95.33 % de precisión (143/150 instancias correctas).

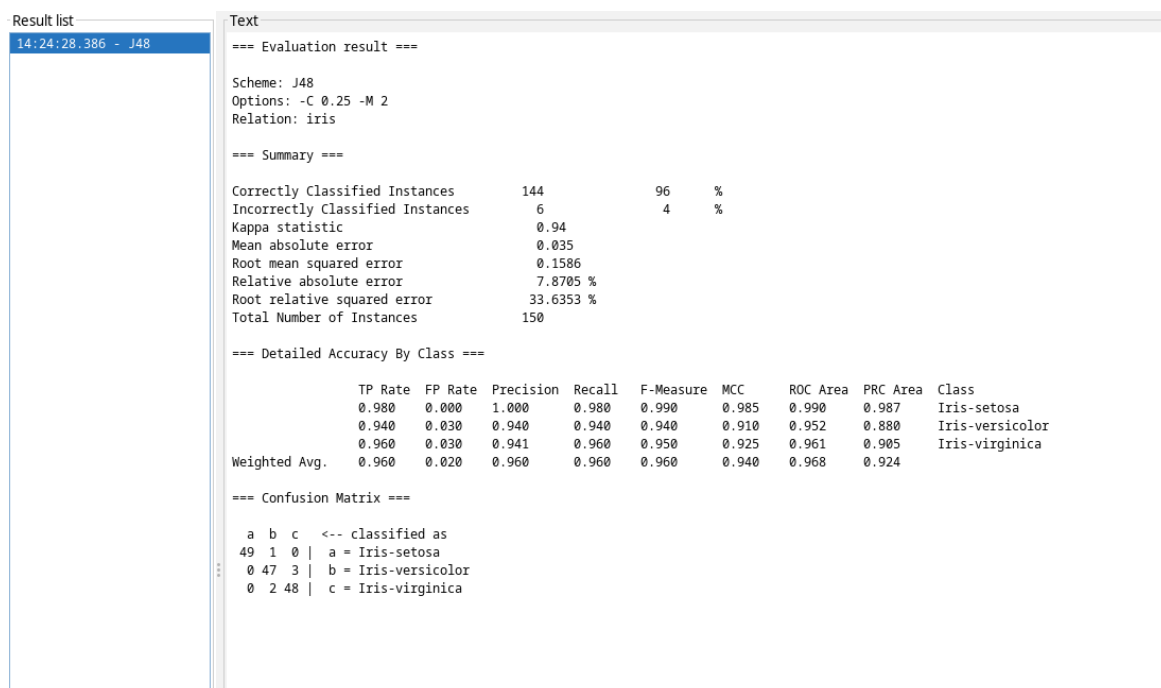


Figura 6: Resultado de J48: 96 % de precisión (144/150 instancias correctas).

Result list
14:35:10.126 - MultilayerP

Text

```

=== Evaluation result ===

Scheme: MultilayerPerceptron
Options: -L 0.3 -M 0.2 -N 500 -V 0 -S 0 -E 20 -H a
Relation: iris

=== Summary ===

Correctly Classified Instances      146          97.3333 %
Incorrectly Classified Instances    4            2.6667 %
Kappa statistic                    0.96
Mean absolute error                0.0327
Root mean squared error            0.1291
Relative absolute error            7.3555 %
Root relative squared error        27.3796 %
Total Number of Instances          150

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall  F-Measure  MCC   ROC Area  PRC Area  Class
      1.000    0.000    1.000    1.000    1.000    1.000    1.000    1.000    Iris-setosa
      0.960    0.020    0.960    0.960    0.960    0.940    0.996    0.993    Iris-versicolor
      0.960    0.020    0.960    0.960    0.960    0.940    0.996    0.993    Iris-virginica
Weighted Avg.    0.973    0.013    0.973    0.973    0.973    0.960    0.998    0.995

=== Confusion Matrix ===

  a  b  c  <-- classified as
50  0  0 | a = Iris-setosa
 0 48  2 | b = Iris-versicolor
 0  2 48 | c = Iris-virginica

```

Figura 7: Resultado de Multilayer Perceptron: 97.33 % de precisión (146/150 instancias correctas), el mejor de los tres.

4.4 Comparación de resultados

Reuniendo las métricas de los tres algoritmos:

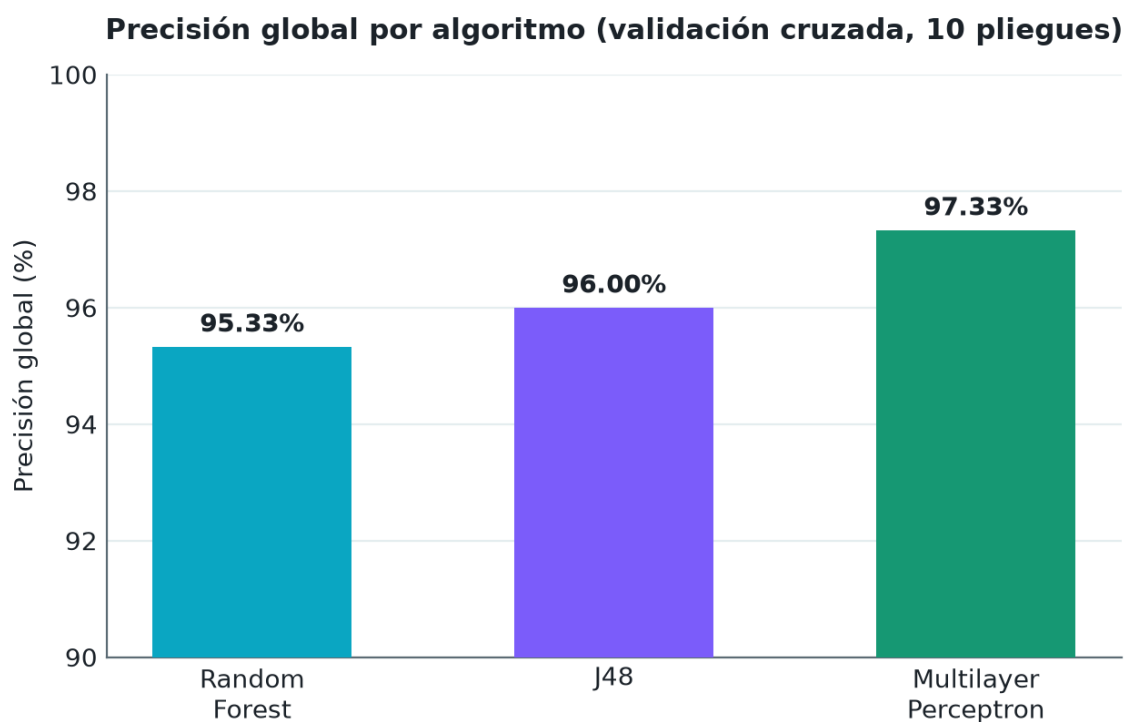


Figura 8: Precisión global de los tres algoritmos evaluados con validación cruzada de 10 pliegues.

La diferencia entre los tres es pequeña (menos de 2 puntos porcentuales), pero la matriz de confusión de cada uno revela *dónde* se equivoca cada algoritmo:

Errores de clasificación por especie y algoritmo

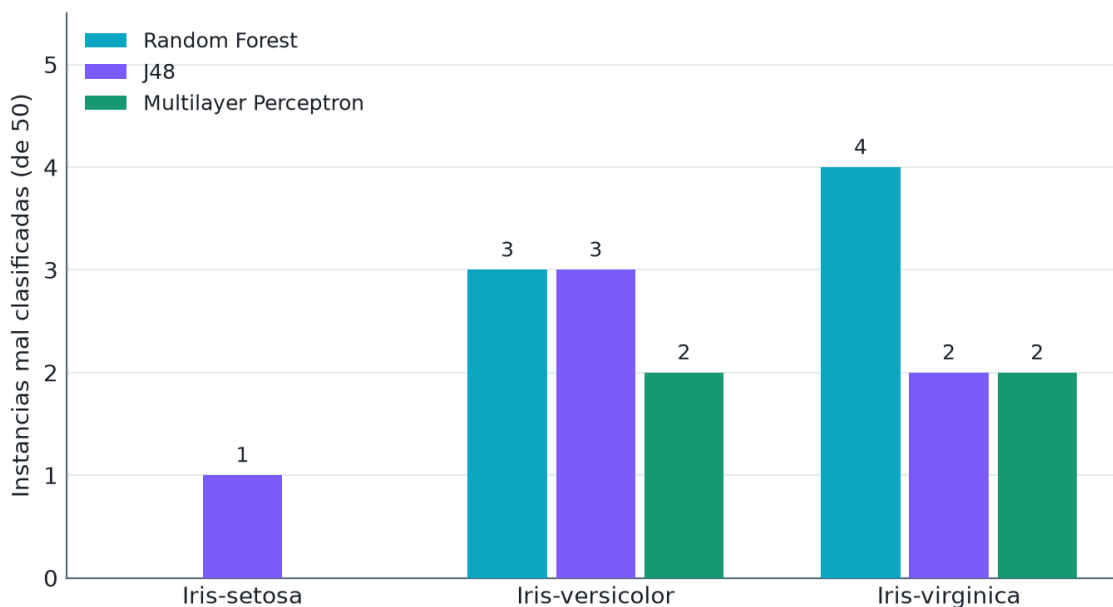


Figura 9: Errores de clasificación por especie. Ningún algoritmo confunde jamás *Iris-setosa* con otra especie; todos los errores ocurren entre *versicolor* y *virginica*.

Esto no es casualidad: el propio dataset documenta que *setosa* es linealmente separable de las otras dos especies, mientras que *versicolor* y *virginica* no lo son entre sí. Los tres algoritmos “descubrieron” ese mismo patrón de forma independiente.

4.5 🌲 Visualización del árbol J48

De los tres algoritmos, solo J48 se puede dibujar como un árbol simple (Random Forest son 100 árboles combinados, y la red neuronal son pesos numéricos, no reglas). Usé el componente **GraphViewer** conectado a la salida **graph** de J48 para visualizarlo:

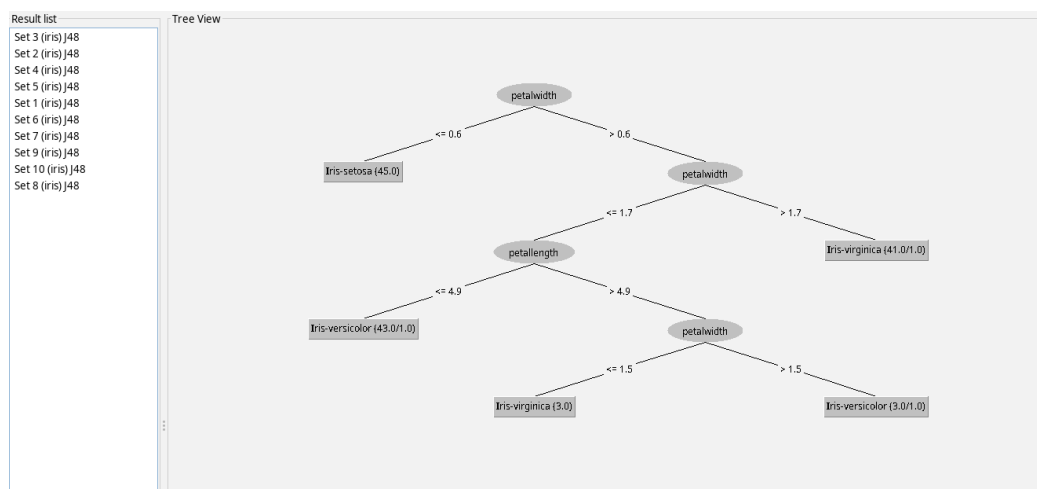


Figura 10: Árbol de decisión generado por J48 en uno de los 10 pliegues de validación cruzada.

El árbol confirma exactamente lo que ya había visto en la correlación de atributos: **nunca usa sepalength ni sepalwidth** para decidir – todas sus preguntas son sobre **petalwidth** y **petallength**. La primera pregunta ($\text{petalwidth} \leq 0.6$) separa *setosa* del resto sin ningún error; las preguntas siguientes van afinando la frontera entre *versicolor* y *virginica*.

5 ⚠ Hallazgo técnico

⚠ Hallazgo técnico

Intenté conectar la salida **graph** también en Random Forest y en Multilayer Perceptron para visualizarlos igual que a J48, pero esa opción de conexión **no aparece** en el menú contextual de ninguno de los dos (solo ofrecen **batchClassifier**, **text** e **info**).

Explicación: la visualización tipo árbol solo está disponible para clasificadores que implementan la interfaz **Drawable** de Weka como un único árbol. Random Forest son 100 árboles combinados (no uno solo) y Multilayer Perceptron es una red de pesos numéricos – ninguno de los dos se puede representar como un árbol de decisión simple, así que Weka correctamente no ofrece esa opción para ellos.

6 📄 Evidencia de ejecución

📄 Evidencia de ejecución

Ejecuté el pipeline completo de principio a fin sin errores: los 12 componentes del flujo (carga, asignación de clase, validación cruzada, 3 clasificadores, 3 evaluadores, 3 visores de texto y el GraphViewer) terminaron correctamente.

Status	Log			
Component	Parameters	Time	Status	
[KnowledgeFlow]		-	OK.	
ArffLoader		-	Finished.	
ClassAssigner		-	Finished.	
CrossValidationFoldMaker		-	Finished.	
RandomForest	-P 100 -I 100 -num-slots 1 -K 0 -M 1.0 -V 0.001 -S 1	-	Finished.	
J48	-C 0.25 -M 2	-	Finished.	
MultilayerPerceptron	-L 0.3 -M 0.2 -N 500 -V 0 -S 0 -E 20 -H a	-	Finished.	
GraphViewer2		-	Finished.	
ClassifierPerformanceEvaluator2		-	Finished.	
ClassifierPerformanceEvaluator		-	Finished.	
TextViewer2		-	Finished.	
TextViewer		-	Finished.	
ClassifierPerformanceEvaluator3		-	Finished.	
TextViewer3		-	Finished.	

Figura 11: Registro de estado (Status) de KnowledgeFlow: todos los componentes del pipeline terminaron con *Finished*.

7 ✅ Conclusiones

- ✅ Construí un pipeline visual completo de ciencia de datos en KnowledgeFlow, desde la carga de datos hasta la comparación de múltiples modelos, sin escribir una sola línea de código.
- ✅ La validación cruzada de 10 pliegues me permitió comparar los algoritmos de forma honesta, evaluando cada uno con datos que nunca usó para entrenar.
- ✅ **Multilayer Perceptron** obtuvo la mejor precisión (97.33 %), seguido de **J48** (96 %) y **Random Forest** (95.33 %) – aunque la red neuronal fue notablemente más lenta de entrenar.

- ✓ Las estadísticas descriptivas (correlación de atributos), el árbol de J48 y las matrices de confusión de los tres algoritmos contaron exactamente la misma historia de forma independiente: el tamaño del pétalo es lo que realmente distingue a las especies, y *Iris-setosa* es trivialmente separable mientras que *versicolor* y *virginica* se solapan.
- ✓ Entendí una limitación real de las herramientas de visualización de modelos: no todo algoritmo se puede dibujar como árbol, y reconocer por qué (ensamble vs. red neuronal vs. árbol único) es tan importante como saber usar la herramienta.

8 Referencias

- Weka – <https://www.cs.waikato.ac.nz/ml/weka>
- Fisher, R.A. (1936). *The use of multiple measurements in taxonomic problems* – fuente original del dataset Iris.